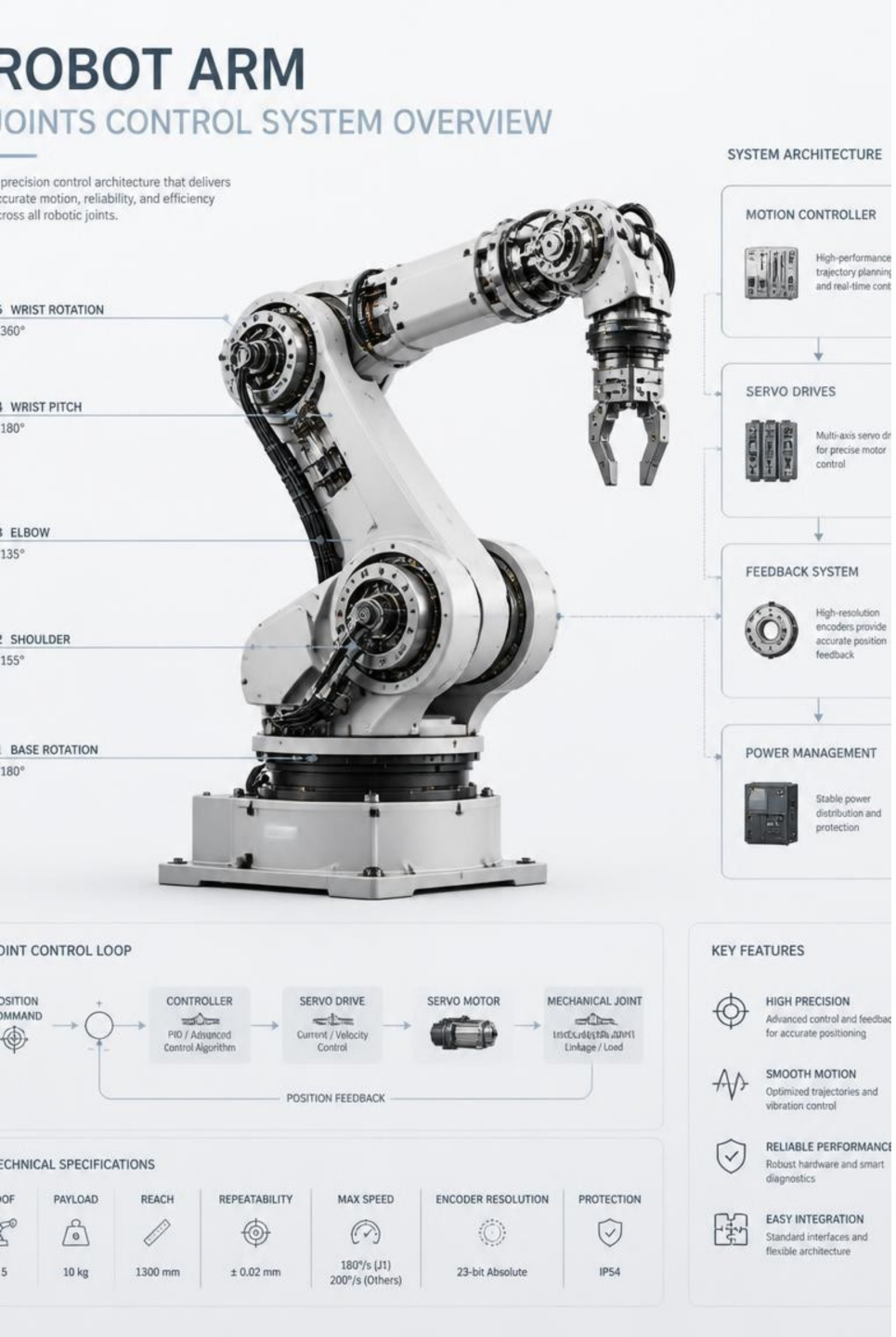
روبوت المحادثة والذكاء الاصطناعي — الجزء الأول

اسم المقرر: أنظمة تشغيل الروبوتات | **الأسبوع الحادي عشر** | مدة الشريحة: 2 ساعة



د. سلطان النفيعي - 2026



مراجعة سريعة — ماذا تعلمنا الأسبوع الماضي؟

01

التحكم في مفاصل الذراع

فهم بنية المفاصل وكيفية التحكم فيها برمجياً

02

أوامر ROS للمفاصل

استخدام أوامر ROS لإرسال إشارات التحكم

03

أداة rqt للتحكم الرسومي

الواجهة الرسومية للتحكم والمراقبة

04

للمفاصل ومسارات الحركةSliders

ضبط الزوايا وتحديد مسارات الحركة البسيطة

د. سلطان النفيعي - 2026

ما سنتعلمه اليوم

🤖 الروبوتات الاجتماعية

تعريف ومفاهيم أساسية

⚙️ المكونات

بنية الروبوت الاجتماعي

📝 لغة AIML

قواعد المحادثة الذكية

🐍 PyAIML

مكتبة Python التطبيقية

💡 أمثلة عملية

تطبيقات وكود حقيقي



ما هي الروبوتات الاجتماعية؟ — Social Robots

تفاعل طبيعي مع البشر

روبوتات مصممة للتواصل الإنساني بأسلوب سلس وطبيعي

فهم المشاعر والاستجابة لها

تعرّف على تعابير الوجه والنبرة الصوتية وتستجيب بمشاعر مناسبة

تتكلم وتستمع وتتعلم

قدرات متكاملة للصوت والاستماع والتعلم المستمر

تستخدم ROS للتحكم والتواصل

منصة ROS تمكّن التكامل بين جميع أنظمة الروبوت

أمثلة روبوتات اجتماعية مشهورة



Jibo

الروبوت المنزلي الودود للعائلة



Nao

— الروبوت التعليمي Aldebaran الصغير



Pepper

— روبوت SoftBank Robotics لخدمة العملاء



Sophia

— أشهر روبوت Hanson Robotics اجتماعي في العالم

تطبيقات الروبوتات الاجتماعية



رعاية المسنين 🧓

مرافقة وتقديم الدعم العاطفي والمساعدة اليومية



تعليم الأطفال 🏫

تفاعل تعليمي ممتع يحفز التعلم والاستكشاف



مساعدة منزلية 🏠

إدارة المهام المنزلية والتذكير بالمواعيد



استقبال الفنادق 🏨

ترحيب الضيوف وتقديم المعلومات السياحية



خدمة العملاء 🛍️

الإجابة على الأسئلة وإرشاد العملاء

مكونات الروبوت الاجتماعي

1 نظام رؤية

كاميرات لرصد البيئة وتعرّف الوجوه

2 نظام صوت

ميكروفون ومكبرات للتواصل الصوتي

3 نظام محادثة

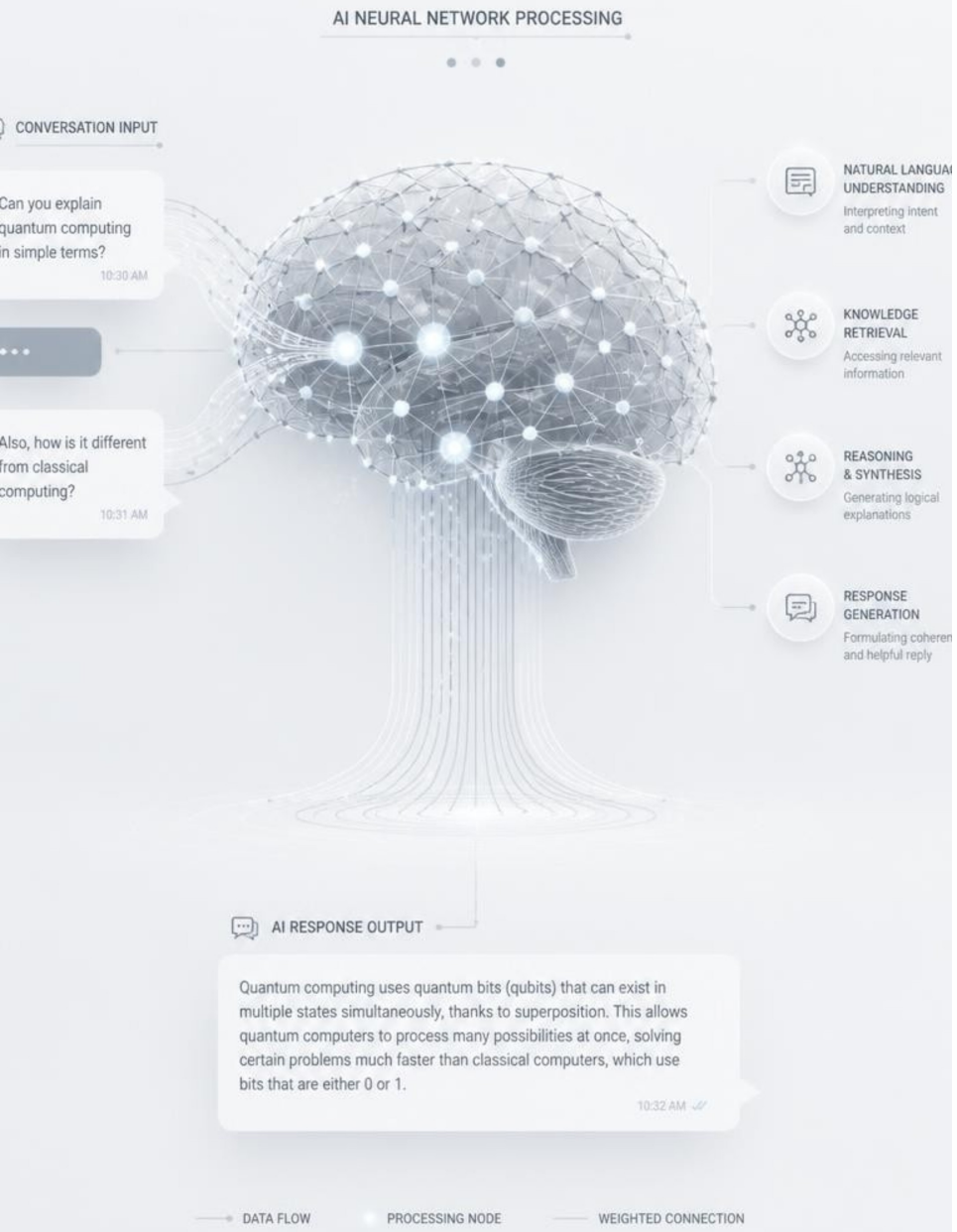
لمعالجة اللغة والردودChatbot / AI

4 نظام حركة وتعبيرات

محركات للحركة والإيماءات وتعبيرات الوجه



نظام المحادثة — Conversation System



🧠 عقل الروبوت الذكي

المركز الفكري الذي يدير كل تفاعل

👂 يفهم الكلام المدخل

معالجة النصوص وتحليل المعنى

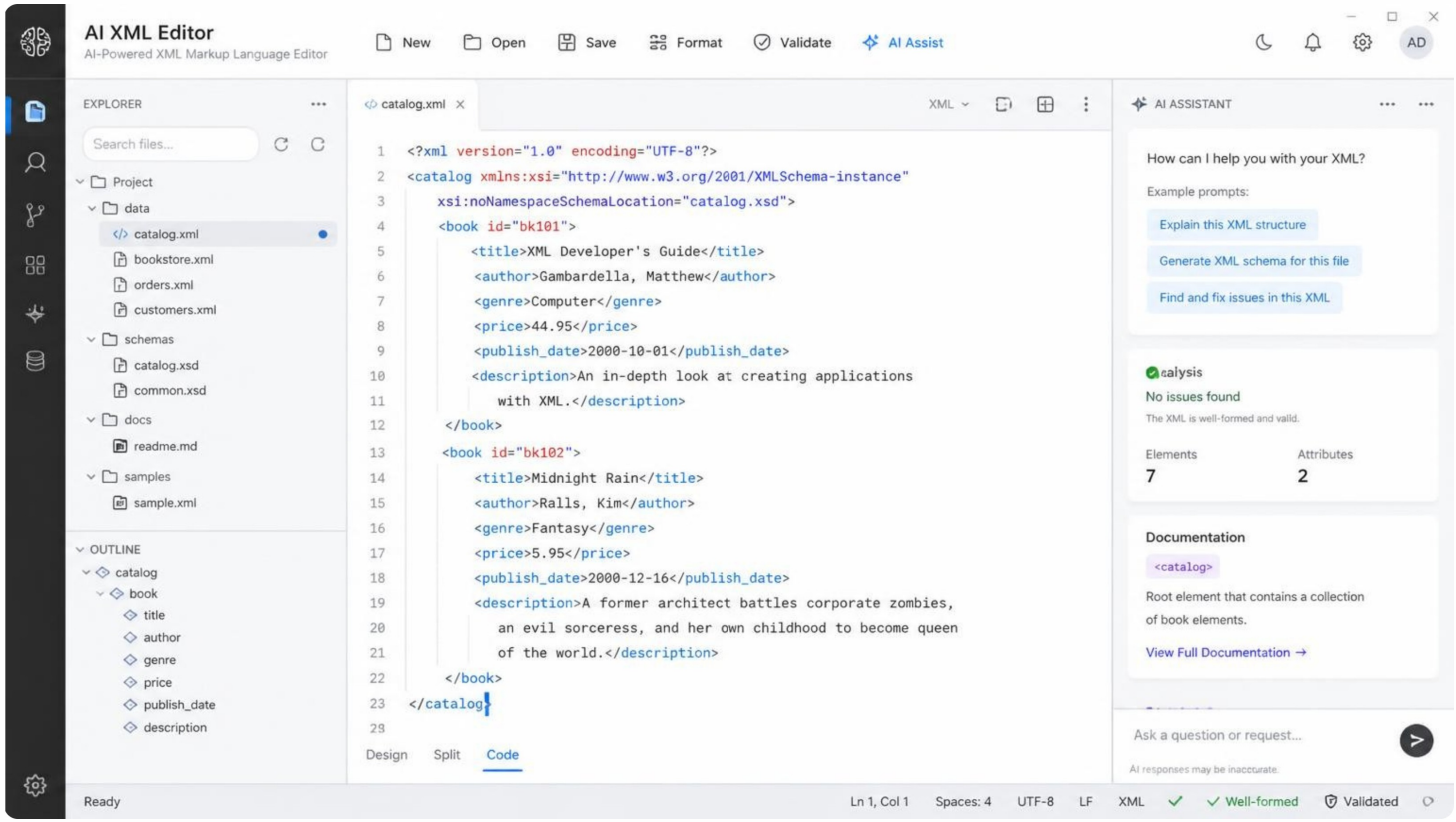
💬 يولّد ردوداً مناسبة

اختيار أنسب رد بناءً على السياق

📚 يتعلم من الحوارات

تحسين مستمر من التفاعلات السابقة

ما هي AIML؟



AIML

Artificial Intelligence Markup Language

لغة XML مصممة خصيصاً لكتابة قواعد المحادثة الذكية

بسيطة وفعّالة

سهلة التعلم وقابلة للتوسع — مفتوحة المصدر ومجانية تماماً

تاريخ AIML

1

1966

برنامج **ELIZA** — أول تجربة محادثة آلية مع الإنسان

2

2001

طوّر **Richard Wallace** لغة AIML مستوحاةً من ELIZA

3

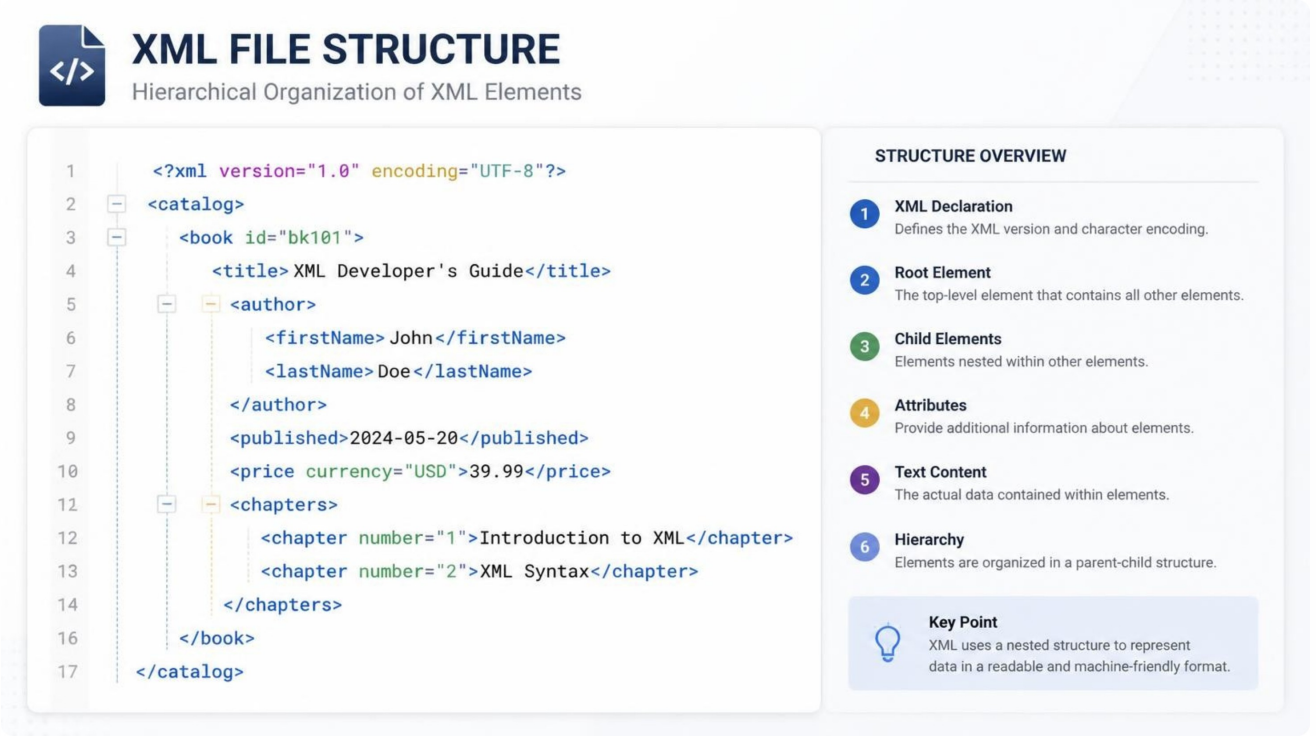
اليوم

معيار مفتوح مستخدم في مشاريع روبوتية وذكاء اصطناعي عالمية

هيكل AIML — الملفات

الوحدة الأساسية: Category

كل ملف AIML يتكون من مجموعة Categories — وكل Category تحتوي على عنصرين أساسيين:



Template

رد الروبوت

Pattern

ما يقوله المستخدم

عنصر Category — وحدة القاعدة

1

Pattern — النمط

`<pattern/>╔╠ᴰ ᵒ _<pattern>`

ما يكتبه أو يقوله المستخدم للروبوت

2

Template — القالب

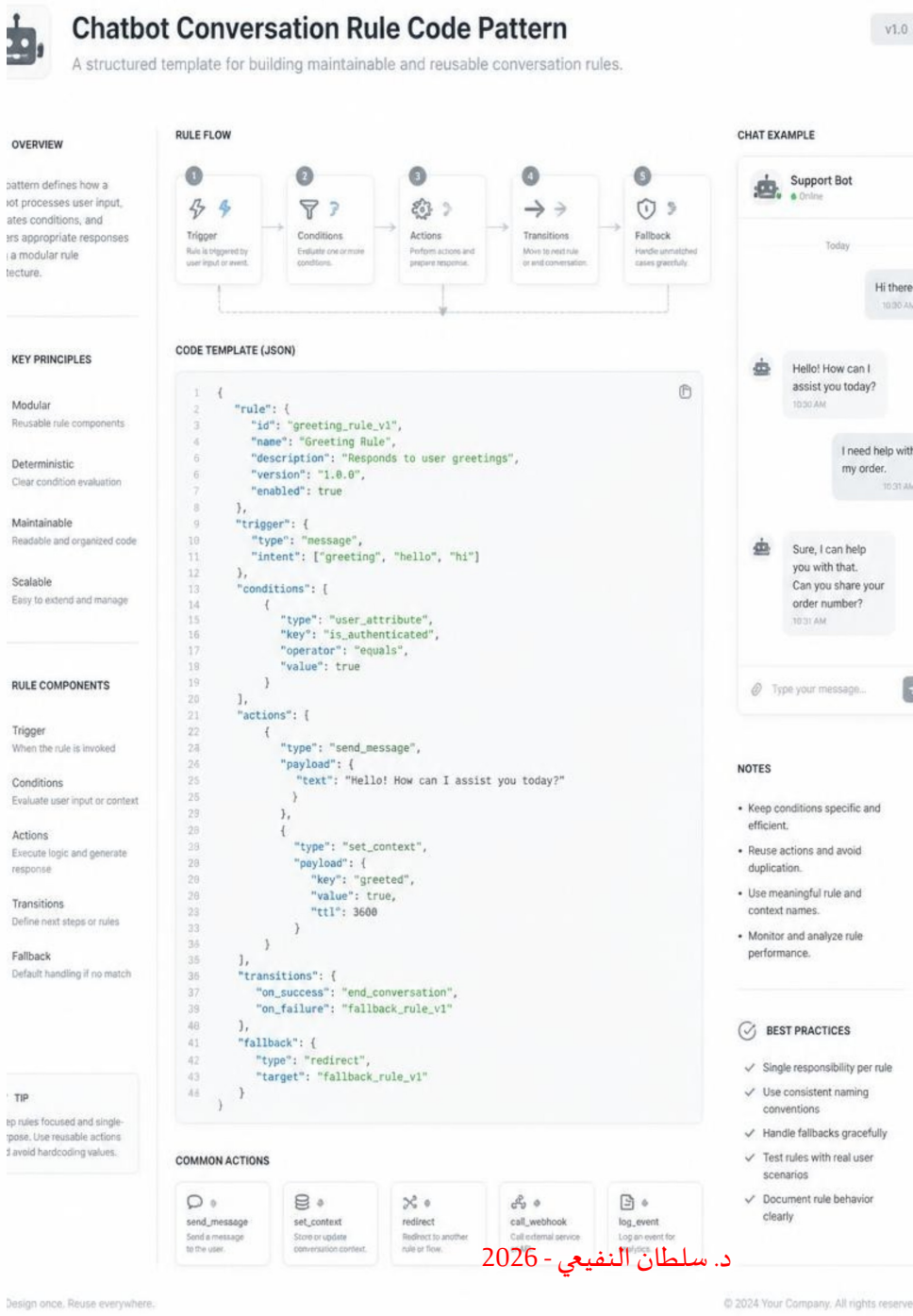
`<template/>!ـ╠ ˋ -ך<template>`

الرد الذي يُرجعه الروبوت تلقائياً

3

قابلة للتوسع

يمكن إضافة آلاف القواعد لتغطية كل مواقف الحوار



د. سلطان النفيعي - 2026

مثال كامل — قواعد بسيطة

Pattern (ما يقوله المستخدم)	Template (رد الروبوت)
مرحبا	أهلاً وسهلاً!
كيف حالك	أنا بخير، شكراً!
اسمك	اسمي روبوت ROS
وداعاً	مع السلامة!

أنماط متقدمة — Wildcards

***** — كلمة أو أكثر

يطابق أي عدد من الكلمات

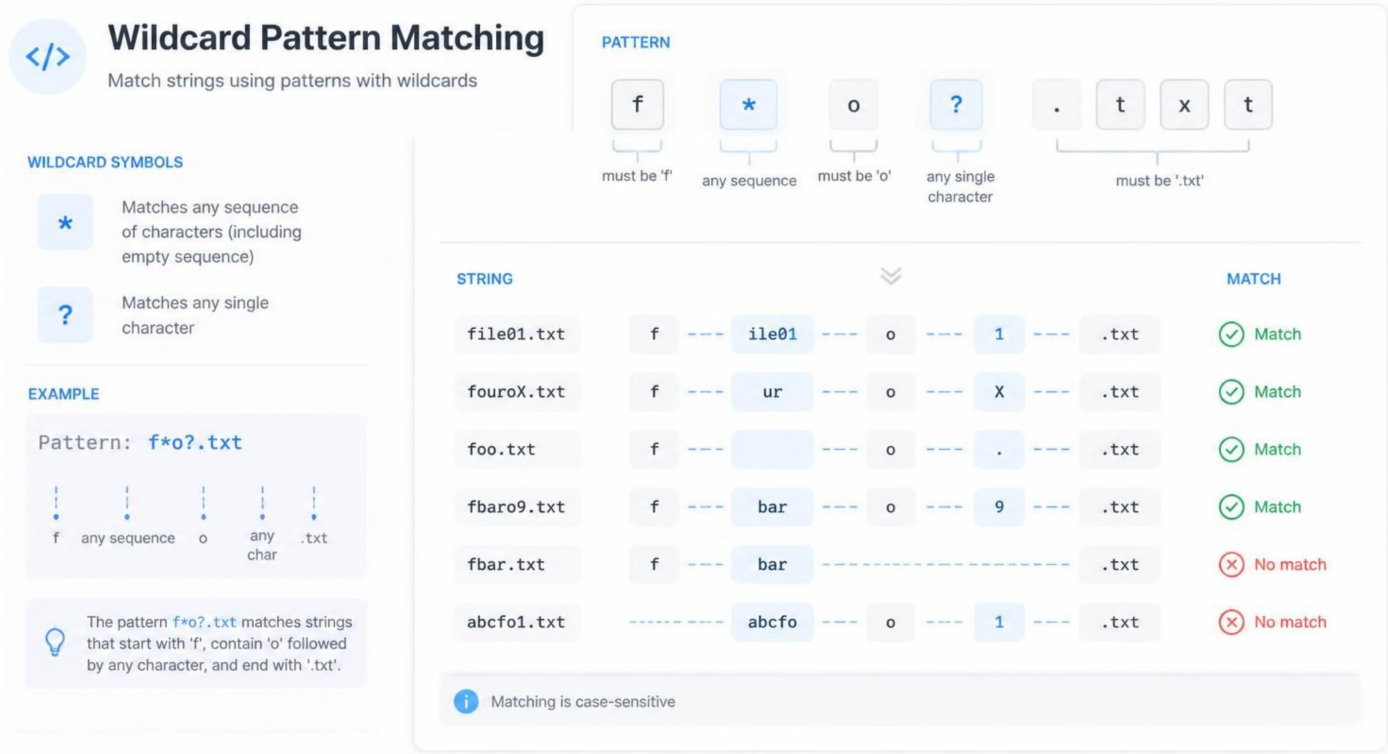
_ — كلمة واحدة

يطابق كلمة واحدة بالضبط

— صفر أو أكثر

يطابق حتى الجملة الفارغة

مثال: `* ╔ ┌ ┐ ᵀ _ ｡` يطابق *"أخبرني ما اسمك"*



أمثلة Wildcards



→ "اسمي روبوت" — يطابق أي جملة تسأل عن الاسم

→ "أنا هنا معك" — يرد على أي سؤال يبدأ بـ "أين"

→ "عمري سنة واحدة" — يغطي صياغات متعددة للسؤال ذاته

Wildcards تجعل المحادثة أكثر مرونة وطبيعية — قاعدة واحدة تغطي حالات لا تُحصى

السياق في That —AIML

كيف يعمل That؟

يتذكر الروبوت آخر رد أرسله ويستخدمه لتحديد السياق الصحيح للمحادثة.

سؤال الروبوت

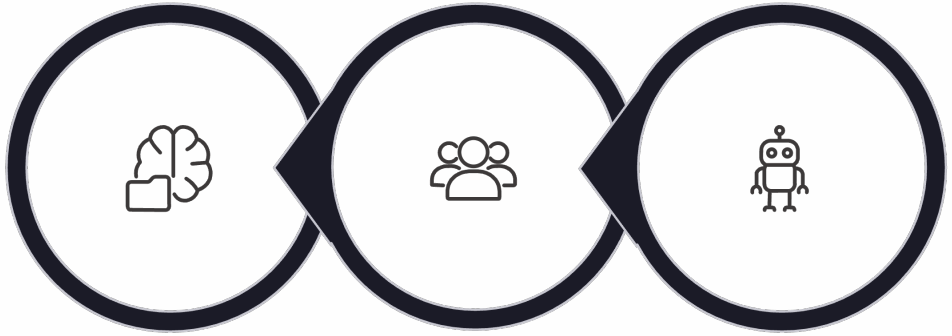
"هل تحب البرمجة؟"

رد المستخدم

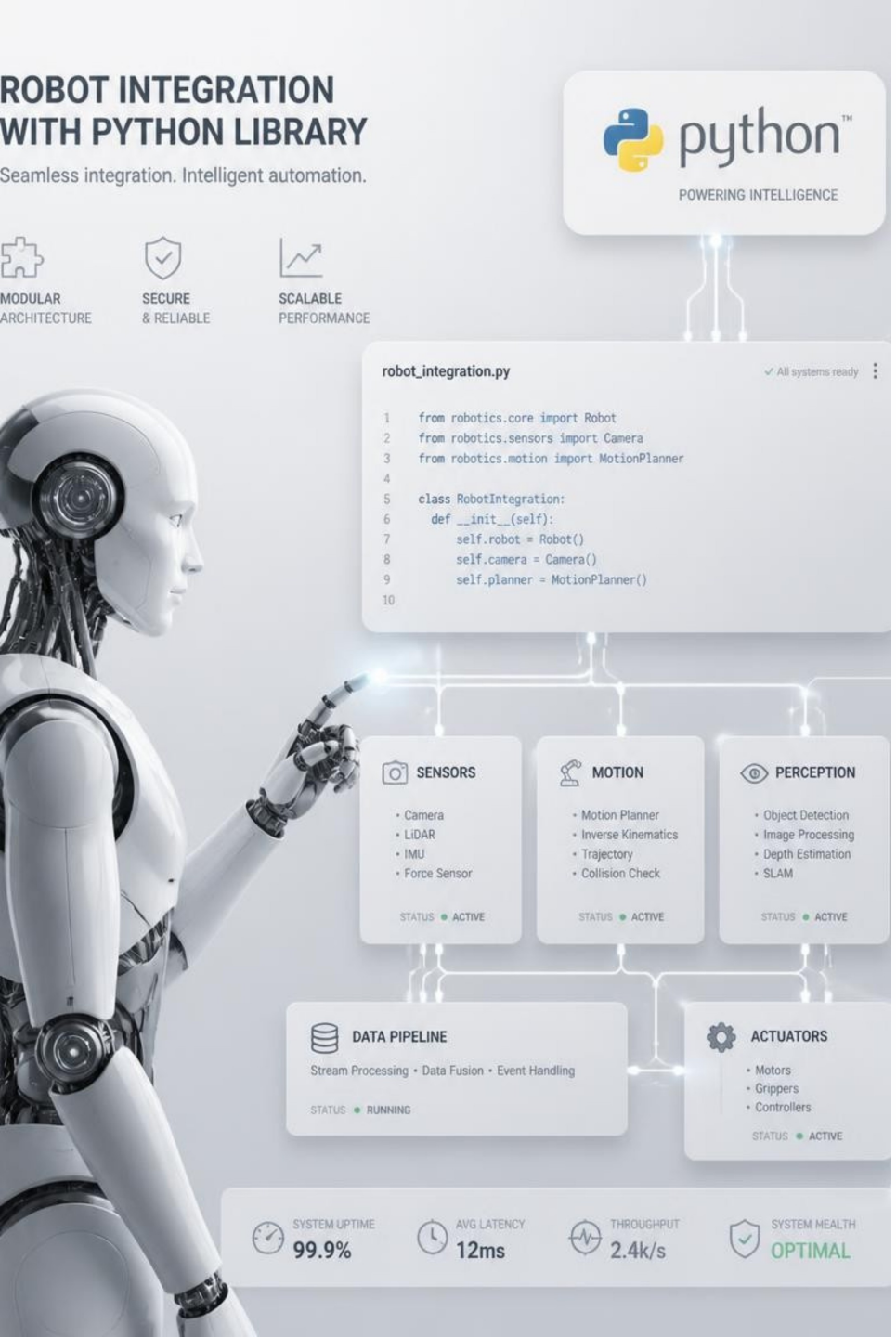
"نعم"

رد الروبوت الذكي

"رائع! البرمجة مهارة رائعة"



الروبوت يسأل　　المستخدم يرد　　الروبوت يتذكر



PyAIML — مكتبة Python

تطبيق AIML لـ Python

تنفيذ كامل لمواصفات AIML بلغة Python الحديثة

تكامل مع ROS

تعمل بسلاسة مع أنظمة ROS للروبوتات

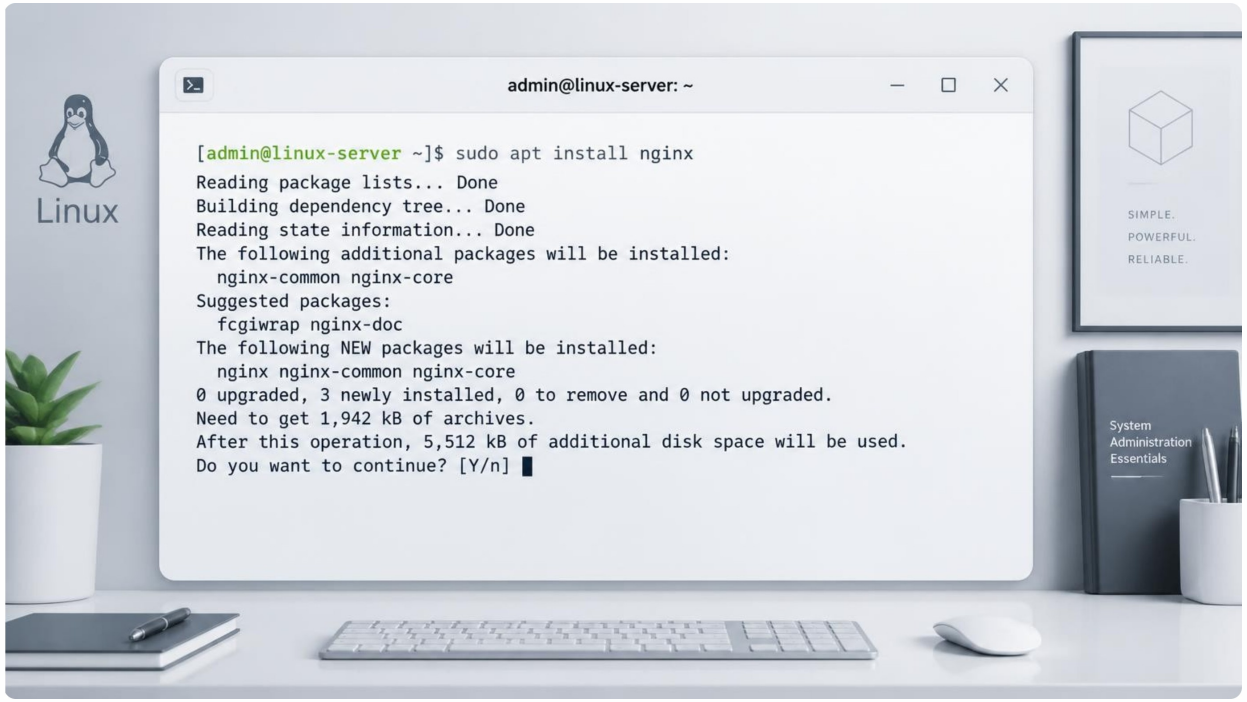
مجانية ومفتوحة

مصدر مفتوح متاح للجميع بدون قيود

سهلة الاستخدام

واجهة برمجية بسيطة وموثقة بشكل جيد

تثبيت PyAIML



طرق التثبيت

يمكن تثبيت مكتبة PyAIML بطريقتين حسب بيئتك:

عبر pip

```
pip install aiml
```

عبر apt (Linux/WSL)

```
sudo apt install python3-aiml
```

يُنصح مستخدمي WSL2 باستخدام الأمر الثاني للتوافق الكامل مع بيئة Linux

إنشاء Kernel — الخطوة الأولى

استيراد المكتبة

```
import aiml
```

إنشاء Kernel

```
kernel = aiml.Kernel()
```

Kernel جاهز للعمل

عقل الروبوت مُهيّأ ويحمل القواعد ويولد الردود

تحميل قواعد AIML

طريقتان للتحميل

تحميل ملف AIML

```
kernel.learn("rules.aiml")
```

يحمّل قواعد XML مباشرة

تحميل Brain محفوظ

```
kernel.loadBrain("brain.brn")
```

أسرع بكثير في التحميل

يمكن تحميل عدة ملفات AIML لتوسيع قاعدة المعرفة



إرسال واستقبال — التفاعل

01

إرسال النص للـ Kernel

```
response =
kernel.respond("╔╠ᴰᵒ_")
```

02

مطابقة النص مع القواعد

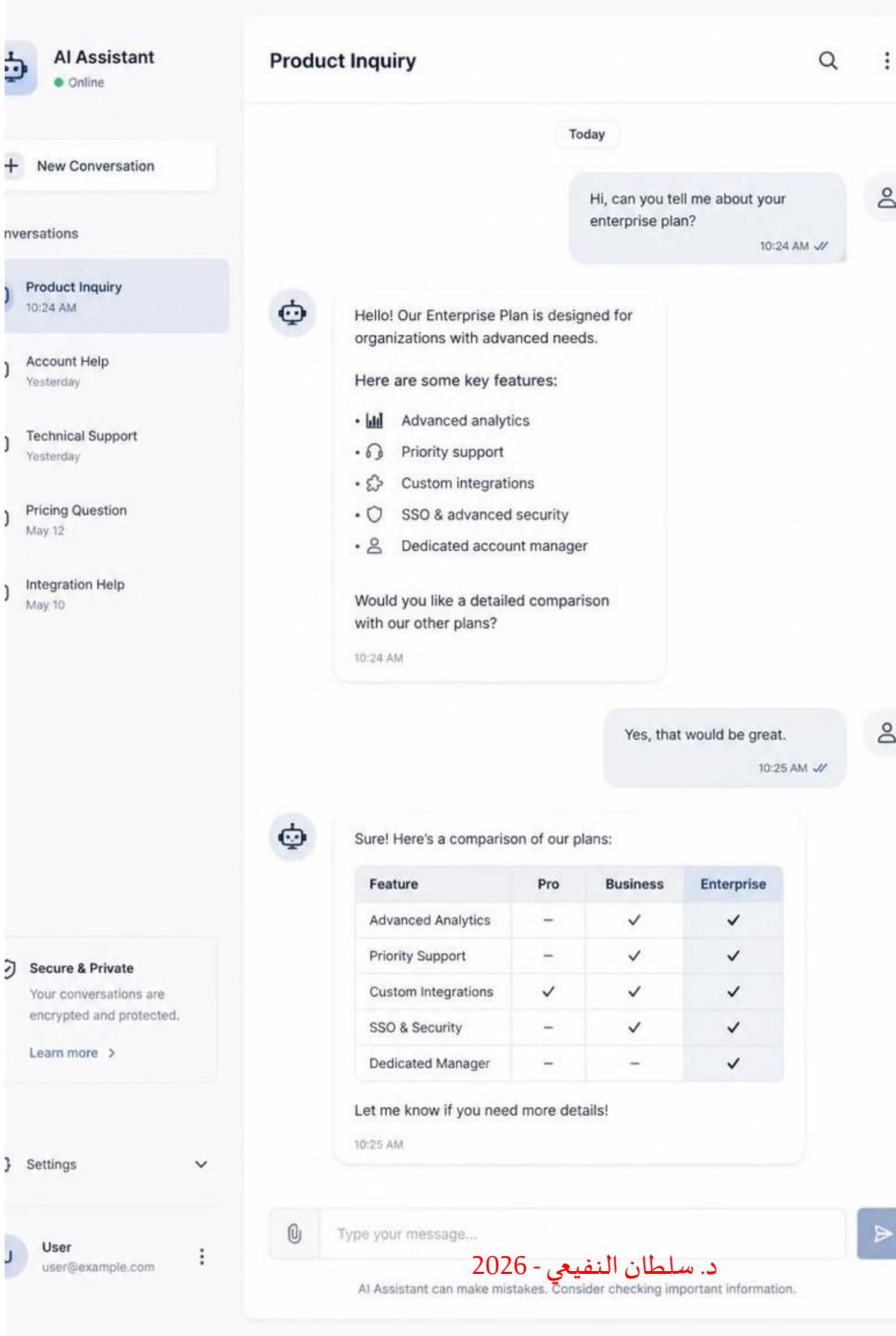
يبحث الـ Kernel عن Pattern المناسب في قواعد AIML

03

طباعة الرد

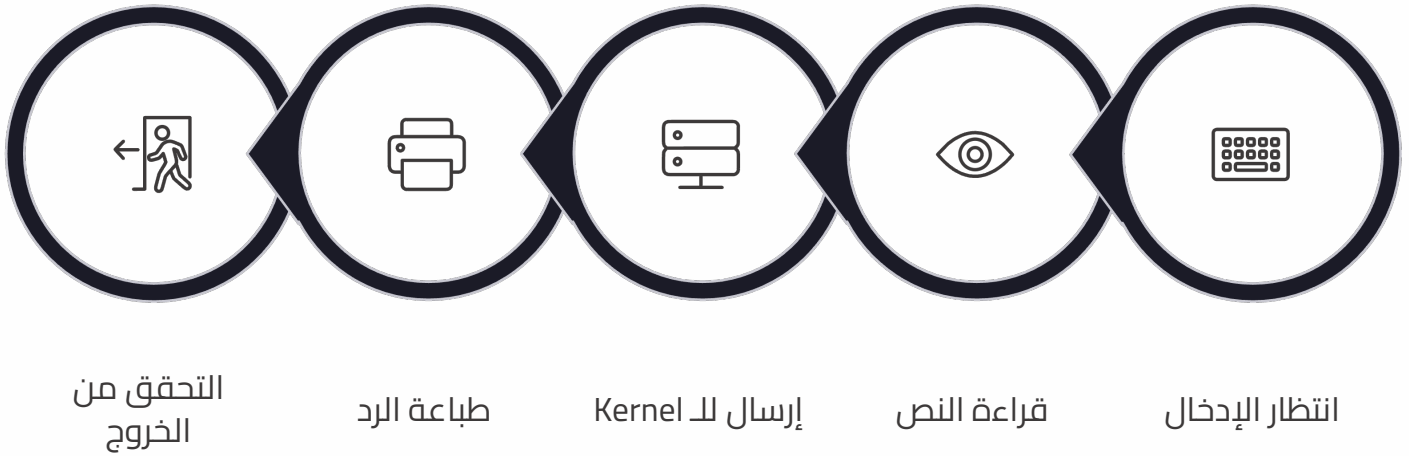
```
print(response)
```

يعيد Template المرتبط بالقاعدة



د. سلطان النفيعي - 2026

حلقة المحادثة — Chat Loop



هيكل الحلقة

```
while True:
    text = input(" :╥~╕")
    if text == "exit:"
        break
    response =
kernel.respond(text)
    print ,":╚/╢/ᴺ")response(
```

حلقة لا نهائية تنتظر المستخدم وترد عليه فوراً، وتنتهي عند كتابة `exit`

ملف AIML كامل — مثال عملي

```
<"8-FTU"=gnidocne "1.0aiml version=">

<category>
<pattern/>╔╠ᴰᴼ_<pattern>
<template/>Ọ.╗ᴷˋ╔ᵁ_ ˋˣ^╔ᵀ╕ ،ˣ¸ !´/ᵀ/ ´⁻╕<template>
<category/>

<category>
<pattern/>.ᵀ╗<pattern>
<template/>!.ˣ˘ˋ ˀᴼ、╔^╗ .~ᴷ、ᵁ~ İROS ╚/╣/ᴺ .ᵀ╗<template>
<category/>

<category>
<pattern/>╔ˋ╗ʲ/<pattern>
<template/>.╔、╚╗ᴺ ╔-/~ .^ ،-_╗╕ !╝-´ᵁ^╗ ˋ_<template>
<category/>

<aiml/>
```

يبدأ كل ملف AIML بتعريف النسخة والترميز، ويحتوي على Categories متعددة، وينتهي بإغلاق الوسم.

سكربت Python كامل

خطوات التشغيل

استيراد المكتبة

```
import aiml
```

إنشاء Kernel وتحميل القواعد

قاعدة المعرفة جاهزة

تشغيل السكربت

```
python3 chatbot.py
```

```
import aiml

 Ò╔ʸ~╣ #Kernel
kernel = aiml.Kernel()

ᴷ˙╗/ˌ^╗ ˇˣ_ᴱ╗ #
kernel.learn("rules.aiml")

╜ŋʲ╔ᴱ_^╗ ╜ˌˇᴰ #
print ╬╔ˌ╗ !ᵠ-╔ɻ ╚/╣/ᵒ^╗")exit("ㄒ/ᵒᴵˇ^
while True:
    text = input(" :╥~╕")
    if text.lower() == "exit:"
        print("!╔˙╗ʲ/")
        break
    response = kernel.respond(text)
    print ,":╚/╣/ᴺ")response(
```

حفظ Brain — تسريع التحميل

لماذا نحفظ الـ Brain؟

ملف `.brn` ثنائي يُحمَّل أسرع بكثير من ملفات AIML النصية — مثالي لبيئات الإنتاج.

حفظ Brain

```
kernel.saveBrain("bot.brn")
```

تحميل سريع

```
kernel.loadBrain("bot.brn")
```

استخدم Brain في الإنتاج لتوفير وقت التحميل وتحسين الأداء



ردود عشوائية — Random Responses

عدة ردود لقاعدة واحدة

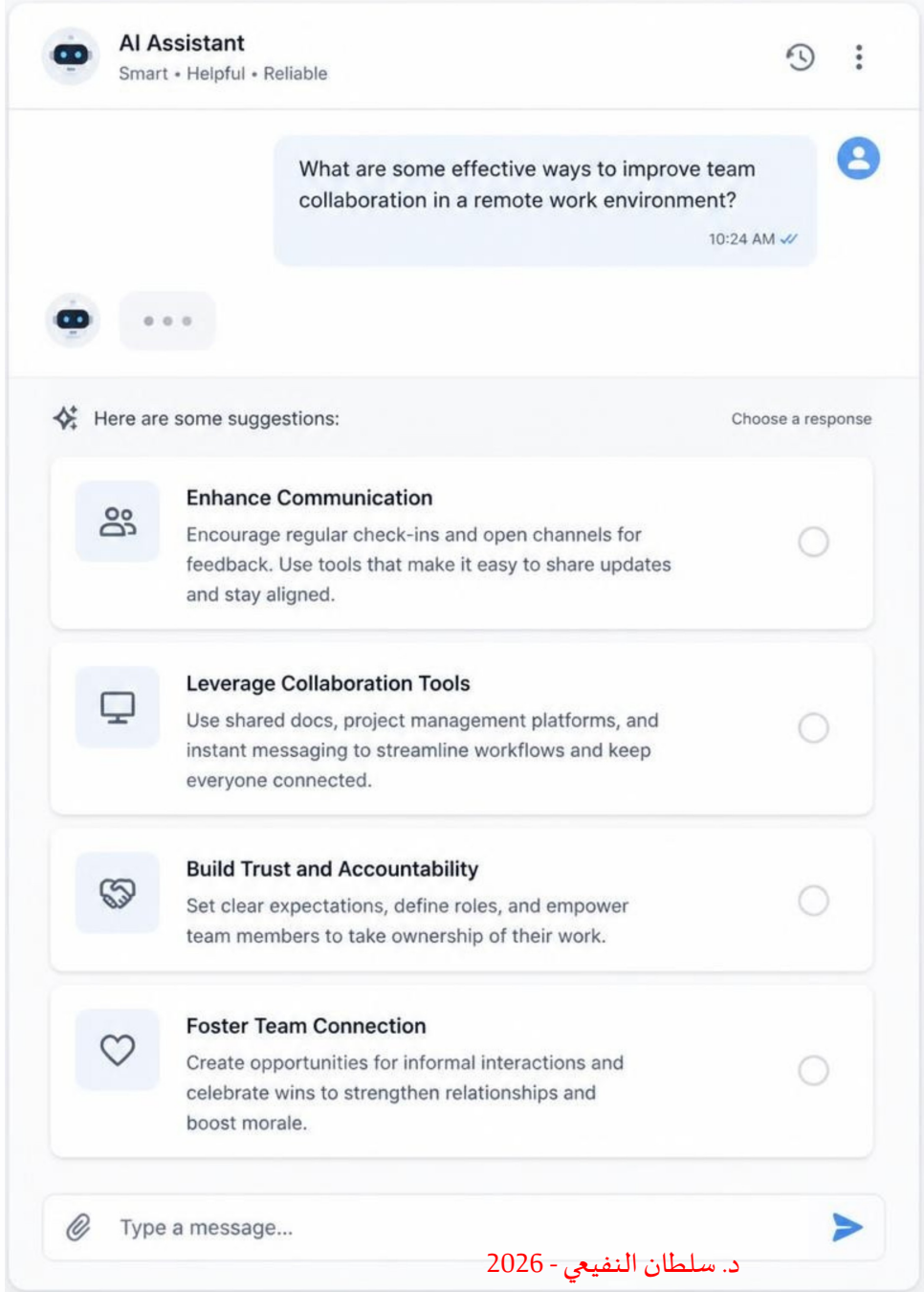
يمكن تعريف أكثر من Template لنفس الـ Pattern

اختيار عشوائي

AIML يختار رداً عشوائياً في كل مرة

محادثة أكثر طبيعية

"كيف حالك" → "بخير" أو "رائع" أو "ممتاز"



د. سلطان النفيعي - 2026

ملخص الدرس — ما تعلمناه اليوم

1 ✓ الروبوتات الاجتماعية

تتفاعل مع البشر بشكل طبيعي وتفهم السياق والمشاعر

2 ✓ لغة AIML

كل قاعدة = Pattern (ما يُقال) + Template (الرد)

3 ✓ Wildcards والسياق

أدوات لجعل المحادثة أكثر مرونة وذكاءً

4 ✓ PyAIML مع Python

Kernel يحمل القواعد، respond() يولّد الردود



أسئلة للمراجعة — اختبر نفسك

1

السؤال الأول

ما هي الروبوتات الاجتماعية؟ وما الذي يميزها عن الروبوتات الصناعية؟

2

السؤال الثاني

ما معنى اختصار AIML وما الغرض منها؟

3

السؤال الثالث

ما الفرق بين Pattern و Template في AIML؟

4

السؤال الرابع

ما وظيفة Wildcard * وكيف يختلف عن _؟

5

السؤال الخامس

كيف تنشئ Kernel في PyAIML وتحمّل عليه قواعد AIML؟



الأسبوع القادم — ماذا سنتعلم؟

🤖 روبوت محادثة في ROS

بناء نظام chatbot متكامل داخل بيئة ROS

🎙️ حزم الصوت

STT (تحويل الكلام لنص) و TTS (تحويل النص لكلام)

🔗 تكامل AIML مع ROS

ربط قواعد AIML مع topics وservicesg في ROS

🔊 نظام صوتي كامل

محادثة صوتية حقيقية من البداية للنهاية

📌 واجب: أنشئ ملف AIML يحتوي على 10 قواعد محادثة مختلفة وجرّبها مع PyAIML